03-18 커스텀 빌드 포털 API 및 리포지토리 구조 논의

커스텀 빌드 포털 API 및 리포지토리 구조 논의

Architecture & API Meeting Summary

커스텀 빌드 포털 API 및 리포지토리 구조 논의

인하우스 개발 포털의 효율적 관리를 위한 리포지토리 구조 재설계 및 배포 파이프라인 정의

리포지토리 관리 구조 재설계

AS-IS: 프로젝트 중심

1 URL (Project)
1 리포지토리 생성

관리 포인트 폭발적 증가 위험

TO-BE: 스테이지 중심

1 Stage (e.g. T1)
1 리포지토리 내 다수 URL 관리

효율적 관리 및 통합 운영 가능

용어 변경: Project → Task Project → Custom Page

배포 통제권 강화

코드 푸시 시 자동 배포되는 기존 방식에서 벗어나, **포털 내 명시적 '배포' 버튼 호출** 방식으로 전환하여 운영 안정성을 확보합니다.

- 실수 방지 및 검수 프로세스
- 버전별 롤백/재시도 용이

신규 배포 파이프라인 워크플로우

1. **코드 업로드** — Git Push 시 포털에 '업로드 완료' 상태 표시
2. **배포 요청** — 운영자가 포털에서 '배포' 버튼 클릭 (API 호출)
3. **S3 및 미리보기** — S3 업로드 성공 후 화면 미리보기 제공
4. **최종 승인** — 운영자 최종 확인 후 서비스 즉시 적용

라우팅 및 활성화 제어

```
// Routing Map JSON Structure
"{
  "path": "/custom/page-1",
  "enabled": true,
  "version": "v1.0.2"
"}"
```

"상태 값을 false로 변경하여 API를 호출하면 허브에서 기본 페이지로 대체 노출함"

주요 API 개발 현황

업로드 및 라우팅 API	Spec Ready
CI/CD 웹훅 (상세 케이스)	Designing
GitHub 템플릿 복제	Portal Direct

테스트 환경 목표: 금요일 ~ 차주 초

Action Items

● 완료 ○ 진행 예정 (TBD)

ian
- ✓ 파일 업로드 및 라우팅 경로 관련 상세 API 스펙 전달
- ○ 포털 API 호출 시에만 배포가 실행되는 시스템 및 API 제작
- ○ URL 패스 단위의 개별 배포 기능 및 라우팅 enabled 로직 구현

sangeun seo(mena)
- ○ 포털 '배포' 버튼 클릭 시 관련 API 호출 기능 구현
- ○ '스테이지' 기반 리포지토리 생성 UI/UX 흐름 수정 및 프로토타입
- ○ '프로젝트' 명칭을 '태스크' 또는 '커스텀 페이지'로 변경 적용

본 회의는 인하우스 개발 포털의 커스텀 빌드 기능 구현을 위해 API 사양을 구체화하고, 리포지토리 관리 구조를 재정의하는 것을 목표로 진행되었습니다. 논의 결과, 기존의 'URL(프로젝트)당 1개 리포지토리' 생성 방식을 '스테이지당 1개 리포지토리'로 변경하고, 이 새로운 구조에 맞춰 배포 파이프라인과 UI/UX 흐름을 조정하기로 합의했습니다.

API 요구사항 및 제공 일정 조율

회의에서는 인하우스 개발 포털의 커스텀 빌드 기능 구현에 필요한 API의 사양과 제공 일정을 구체적으로 논의했습니다. ian은 현재 개발 중인 API 스펙의 진행 상황을 공유하며, 파일 업로드 및 배포, 라우팅 경로 적용과 관련된 상세 스펙은 당일 중으로 전달 가능하다고 밝혔습니다. 반면, GitHub 템플릿 리포지토리를 복제하는 기능은 별도의 API 구현 없이 GitHub API를 직접 활용하여 포털에서 처리하는 것으로 정리되었습니다.

이와 함께 CI/CD 파이프라인의 배포 완료 시점을 포털에 통지하는 웹훅(Webhook) API에 대한 요구사항도 논의되었습니다. sangeun seo(mena)는 단순한 성공 여부 외에, 다양한 실패 원인에 따른 상세한 응답 값과 케이스별 재시도(예: 전체 재시작, 특정 단계부터 재시작)를 처리할 수 있는 스펙 제공을 요청했습니다. ian은 해당 API의 테스트 가능 시점을 금요일 또는 차주 초로 예상했으며, 오늘 중으로 상세 스펙 설계를 마무리하겠다고 답했습니다.

배포 파이프라인 단계별 동작 방식 정의

사용자가 포털에서 프로젝트를 생성하고 배포하는 전체 과정을 단계별로 상세하게 검토하고 동작 방식을 정의했습니다. 현재 포털은 프로젝트 생성 시 배포 관련 옵션 없이 화면 노출 설정만 진행하고, 생성된 프로젝트의 상세 화면 내 '배포 및 운영' 탭에서 실제 배포 파이프라인이 시작되는 구조입니다.

가장 중요한 결정 사항으로, 기존의 코드 푸시(push) 시 자동 배포되던 방식에서 사용자가 포털에서 '배포' 버튼을 눌렀을 때만 지정된 API가 호출되어 배포가 실행되도록 변경하기로 합의했습니다. 이 새로운 흐름은 다음과 같습니다.

1. **코드 업로드**: 개발자가 Git 리포지토리의 메인 브랜치에 코드를 푸시하면, 웹훅이 이를 감지하여 포털에 '코드 업로드 완료' 상태를 표시합니다.
2. **배포 요청**: 운영자(예: 루크)가 포털에서 배포할 버전을 확인하고 '배포' 버튼을 클릭하여 API를 호출합니다.
3. **S3 업로드 및 화면 생성**: 호출된 API는 GitHub의 최신 코드를 S3에 업로드하고, 업로드가 성공하면 포털에 응답을 보냅니다.
4. **미리보기 및 최종 승인**: 포털은 성공 응답을 받아 생성된 화면의 미리보기를 제공합니다. 운영자는 미리보기를 통해 이슈 여부를 확인한 후 최종 '승인' 버튼을 누르면 서비스에 적용됩니다.

이 과정을 위해 'S3 업로드 API'와 '라우트맵 반영(배포) API' 두 가지가 필요하며, ian의 팀에서 이를 개발하기로 했습니다.

리포지토리 관리 구조 재설계: 프로젝트 단위에서 스테이지 단위로

기존 '1 URL(프로젝트) = 1 리포지토리' 구조가 장기적으로 관리할 리포지토리의 수를 폭발적으로 증가시킬 것이라는 문제점에 양측이 공감했습니다. 이에 대한 해결책으로, 리포지토리 생성 단위를 '스테이지(Stage)'로 변경하는 중대한 구조 재설계에 합의했습니다.

새로운 구조는 '1 스테이지 = 1 리포지토리'를 원칙으로 합니다. 즉, 하나의 스테이지(예: T1)에 대한 모든 커스텀 페이지(다수의 URL 경로)의 코드는 단일 리포지토리 내에서 관리됩니다. 이 변경에 따라 포털의 UI/UX 흐름도 수정이 필요합니다. 사용자는 먼저 커스텀을 진행할 '스테이지'를 선택하고, 이때 해당 스테이지에 매핑될 리포지토리가 생성됩니다. 이후 생성된 리포지토리 내에서 개별 URL 경로(패스)에 대한 커스텀 작업을 추가하고 관리하게 됩니다. 또한, 현재 UI에서 '프로젝트'로 지칭되는 개념이 실제로는 개별 'URL/패스' 단위를 의미하므로, '태스크' 또는 '커스텀 페이지' 등 더 명확한 용어로 변경하기로 했습니다.

신규 구조에 따른 배포 및 라우팅 제어 방식 논의

스테이지 단위 리포지토리 구조로 변경됨에 따라, 배포 및 라우팅 제어 방식을 세부적으로 논의했습니다. 단일 리포지토리 내에 여러 URL(패스)의 코드가 공존하므로, 사용자가 수정한 특정 패스만 개별적으로 배포할 수 있는 기능이 필수적임을 확인했습니다. ian은 API를 통해 특정 패스 정보만 받아 해당 부분만 업로드 및 배포하는 것이 기술적으로 가능하다고 밝혔습니다.

또한, 각 패스별로 커스텀 페이지 사용 여부를 켜고 끌 수 있는 '활성화/비활성화' 기능 구현 방안도 논의되었습니다. 최종적으로, 라우팅 설정이 담긴 JSON 파일 내 각 경로 정보에 `enabled: true/false` 와 같은 상태 값을 추가하기로 결정했습니다. 포털에서 비활성화를 요청하면 이 값을 `false` 로 변경하여 API를 호출하고, 프론트엔드(허브)는 이 값을 해석하여 커스텀 페이지 대신 기본 페이지를 노출하게 됩니다. 이는 라우팅 정보를 삭제하고 다시 추가하는 방식보다 상태 관리가 용이하다는 장점이 있습니다. 이 기능은 ian의 팀에서 FE팀과 협의하여 구현을 진행하기로 했습니다.

Action Items

@devops

- [ ] 파일 업로드 및 라우팅 경로 적용 관련 상세 API 스펙 전달예정
- [ ] GitHub API를 활용한 템플릿 레포지토리 복제 방식 확인 후 공유 - [TBD]
- [ ] 샘플 레포지토리
  - [ ] 샘플 파일과 CI/CD 파일을 취합하여 전달 - [TBD]
- [ ] URL 패스 단위의 개별 배포가 가능하도록 API 기능 구현 - [TBD]

- [ ] 개발된 API를 테스트해볼 수 있는 환경 제공 (목표: 금요일 또는 차주 초) - [TBD]
- [ ] 라우팅 맵 JSON에 `true/false` 값을 추가하여 패스별 활성화/비활성화 기능 구현 및 FE팀과 논의 - [TBD]
- [ ] 회의에서 논의된 아키텍처(스테이지 단위 레포, 패스별 관리 등)를 기반으로 API 개발 및 연동을 통한 구체화 진행 - [TBD]

@sangeun seo(mena)

- [ ] 포털에서 깃헙 배포(전체 레포 배포)와 패스별 서비스 적용 분리 [TBD]
- [ ] 현재 '프로젝트'로 명명된 단위를 '태스크' 또는 '커스텀 페이지' 등 새로운 용어로 변경 - [TBD]
- [ ] '스테이지' 선택 시 리포지토리가 생성되는 새로운 UI/UX 흐름으로 수정하고 프로토타입 공유 - [TBD]